

☁️ Accenture Cloud Computing — Tier 3

Core Conceptual Revision • Architectural Patterns • High-Yield Assessment MCQs

1. Serverless Computing ★★★★★

Definition

Serverless computing is a cloud execution model where the cloud provider dynamically manages the allocation, provisioning, and scaling of underlying server infrastructure, allowing developers to focus primarily on application code without worrying about server administration.

Important Core Tenet:

- Serverless does **NOT** mean there are no servers.
- Servers still exist physically in the cloud provider's datacenters.
- The cloud provider completely manages, patches, secures, and provisions them for you.

Architectural Comparison

Traditional Infrastructure	Serverless Model
<pre> Developer ↓ Application ↓ OS ↓ Server (VM / Physical) ↓ Hardware </pre>	<pre> Developer ↓ Function / Application Code ↓ Cloud Provider automatically manages infrastructure, OS, and scaling </pre>

Key Benefits

- **No Server Management:** Zero administrative burden for OS patches, hardware maintenance, or runtime setup.
- **Automatic Scaling:** Elastic scaling from 0 to thousands of concurrent requests seamlessly.
- **Pay-per-Use Models:** Billed strictly for execution duration (measured in milliseconds) and memory consumed; zero cost when idle.
- **Faster Time-to-Market:** Accelerated development lifecycle focusing purely on business logic.

2. FaaS (Function as a Service) ★★★★★

FaaS = Function as a Service. FaaS allows developers to deploy modular, standalone units of business logic (individual functions) that execute ephemeral tasks strictly in response to event triggers or incoming HTTP requests.

Event-Driven Execution Lifecycle

```
graph TD; A[User uploads image (S3 / Blob)] --> B[Event Trigger emitted]; B --> C[Serverless Function executes]; C --> D[Resize & optimize image]; D --> E[Persist to DB & Done (Function terminates)];
```

Industry-Leading FaaS Offerings

- **AWS Lambda** (Amazon Web Services)
- **Azure Functions** (Microsoft Azure)
- **Google Cloud Functions** (Google Cloud Platform)

 **Remember:** FaaS → Run individual functions without provisioning or managing servers.

Serverless vs FaaS: The Relationship

These terms are closely related but not identical. **Serverless** is the overarching architectural umbrella (spanning serverless databases like Aurora Serverless/DynamoDB, serverless storage like S3, and API Gateways), whereas **FaaS** is the specific serverless compute paradigm executing granular functions.

```
graph TD; A[Serverless (Broader Architecture: Compute, Storage, DBs, Queues)] --> B[FaaS (Compute Layer Sub-category)]; B --> C[Functions (Individual code execution units)];
```

3. Microservices Architecture ★★★★★

Definition

Microservices architecture decomposes a software application into a suite of small, autonomous, and independently deployable services organized around distinct business capabilities.

Traditional Monolithic Approach	Microservices Approach
One massive, tightly-coupled codebase containing all modules.	Application — User Service — Payment Service — Order Service — Notification Service

Example: E-Commerce Architecture

```
graph TD; A[Frontend (Web / Mobile App)] --> B[ ]; B --> C[ ]; B --> D[ ];
```

User Service (User Profile DB)	Order Service (Orders DB)	Payment Service (Payment Gateway)
-----------------------------------	------------------------------	--------------------------------------

Advantages & Disadvantages

Advantages	Disadvantages / Trade-offs
• Independent Deployment: Update one service without touching others. • Targeted Scaling: Scale only resource-heavy services. • Team Autonomy: Isolated polyglot development stacks. • Fault Isolation: Failure in payment won't crash browsing.	• Network Overhead: Latency from inter-service calls. • Operational Complexity: Requires advanced orchestration (Kubernetes). • Distributed Systems: Complex data consistency (Sagas/eventual consistency).

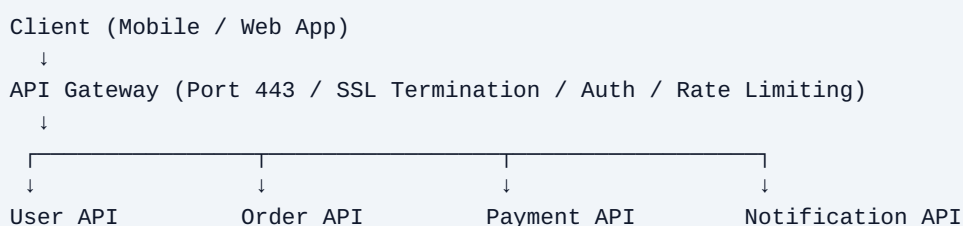
4. Monolith vs Microservices ★★★★★

Dimension	Monolithic Architecture	Microservices Architecture
Codebase	Single unified codebase with shared memory and libraries.	Multiple discrete code repositories by service boundary.
Deployment	Single artifact; entire system must be redeployed for minor fixes.	Independent CI/CD pipelines and isolated containerized deployments.
Database	Typically a single centralized database.	Database-per-service pattern (polyglot persistence).
Scaling	Horizontal scaling duplicates the entire stack.	Granular scaling tailored to specific microservice workload demands.

🧠 **MCQ Quick Reference:** If an organization wants to deploy or scale the Payment service independently without redeploying the entire application → **Microservices Architecture**.

5. API Gateway ★★★★★

An **API Gateway** serves as the centralized single entry point (reverse proxy) for all incoming client requests routing into backend microservices.



Core Functions of an API Gateway

- **Request Routing & Aggregation:** Directs traffic to matching microservices and combines responses.
- **Authentication & Authorization:** Verifies JWT tokens / API keys at perimeter.
- **Rate Limiting & Throttling:** Protects downstream services from DoS attacks and traffic spikes.
- **Request / Response Transformation:** Modifies headers, protocols (e.g., HTTP to gRPC), or payloads.
- **Telemetry & Logging:** Centralized metrics collection for latencies and error rates.

 **Remember:** API Gateway → Unified Entry point and policy enforcement for APIs.

6. Monitoring & 7. Logging in Cloud Systems ★★★★★

Monitoring (System Telemetry & Metrics)	Logging (Discrete Historical Records)
Core Question: "What is the system health right now?" Continuous collection and evaluation of quantitative performance metrics across cloud resources. • CPU & Memory Utilization • Network I/O & Bandwidth • Request throughput, Error rates, & Latency • Uptime & Availability CPU Utilization > 80% ↓ Monitoring system triggers threshold ↓ Automated Alert / Auto Scaling	Core Question: "What specific events occurred in the past?" Time-stamped textual records generated by applications, infrastructure, and security agents. 2026-08-19 14:20:11 UTC [WARN] User authentication failed. IP: 198.51.100.42 Reason: Invalid credentials • Root cause analysis & debugging • Security audits & forensics investigations • Compliance and regulatory traceability

 **Memory Hook:** Monitoring → What is happening? | Logging → What happened?

8. Backup, 9. Disaster Recovery, 10. RPO & 11. RTO ★★★★★

Cloud Backup vs Disaster Recovery (DR)

- **Backup:** The physical or logical copy of data created and stored in isolated storage (e.g., AWS S3, Azure Blob) to restore data after corruption, accidental deletion, ransomware, or disk failure.
- **Disaster Recovery (DR):** The comprehensive end-to-end strategic process, technical infrastructure, and automated orchestration used to restore complete operational functionality of systems, applications, and networks after a catastrophic outage or regional datacenter disaster.

Disaster Recovery Metrics: RPO vs RTO

Metric	Key Focus Question	Measurement Unit	Primary Driver
--------	--------------------	------------------	----------------

RPO (Recovery Point Objective)	"How much data loss can the organization tolerate?"	Time duration (e.g., 5 mins, 1 hr)	Backup frequency and data replication design.
RTO (Recovery Time Objective)	"How quickly must the system/service be restored to operations?"	Time duration (e.g., 15 mins, 4 hrs)	Recovery strategy, automation, and failover architecture.

RPO Timeline Example:

12:00 (Backup) -----> 13:00 (Backup) -----> 14:00 (DISASTER EVENT OCCURS)



If recovered to 13:00 point-in-time: Potential Data Loss = 1 Hour (Target must be within RPO)

🧠 Easy Memory Hook:

- RPO → Point → Data Loss (How much data can we lose?)
- RTO → Time → Downtime (How much time until systems are back up?)

12. Disaster Recovery (DR) Strategies ★★★★★

Strategy	Cost	RTO / RPO	Operational Mechanism
1. Backup & Restore	\$ (Lowest)	Highest (Hours/Days)	Data is backed up to cloud storage. In disaster, new infrastructure is provisioned and data restored from scratch.
2. Pilot Light	\$\$	Moderate (Tens of mins)	Core components (e.g., DB with continuous replication) remain running. Compute servers are spun up only when disaster strikes.
3. Warm Standby	\$\$\$	Low (Minutes)	A scaled-down but fully functional version of the entire environment runs continuously in the secondary region; scaled up upon failover.
4. Active-Active (Multi-Region)	\$\$\$\$ (Highest)	Near Zero	Traffic is actively routed across two or more operational regions simultaneously via global load balancing. Real-time failover.

13. Multi-Cloud & 14. Cloud Bursting ★★★★★

Multi-Cloud Architecture	Cloud Bursting (Hybrid Scaling)
Using cloud computing services and infrastructure from two or more distinct public cloud vendors (e.g., AWS + Azure + GCP). • Pros: Prevents vendor lock-in, optimizes cost by	A hybrid cloud configuration where applications typically run in a private cloud / on-premises data center, but dynamically burst into a public cloud during peak demand spikes. Normal Traffic (10k Users) → Private Cloud Handles Load Peak Traffic (1M Users) → Private Cloud + Public Cloud Burst

choosing best-of-breed services (e.g., AWS EC2 + GCP BigQuery).

- **Cons:** High operational complexity, differing management tooling, complex governance & security.

 **Quick Takeaway:** Private normally + Public cloud during traffic peaks → **Cloud Bursting**.

15. Infrastructure as Code, 16. DevOps, & 17. CI/CD ★★★★★

Infrastructure as Code (IaC)

IaC is the practice of managing, provisioning, and configuring infrastructure resources (networks, VMs, load balancers, databases) using machine-readable definition files rather than manual human configuration via interactive web consoles.

- **Popular Tools:** HashiCorp Terraform (declarative/multi-cloud), AWS CloudFormation, Azure Bicep, Ansible.
- **Key Benefits:** Repeatability, version control (Git), automated auditing, elimination of configuration drift.

DevOps & Continuous Integration / Continuous Delivery (CI/CD)

DevOps unites development and operations teams through automation, cross-functional collaboration, and shared ownership to accelerate software delivery cycles with higher quality.

Continuous Integration (CI)	Continuous Delivery / Deployment (CD)
Developers merge code changes frequently into a centralized shared repository (Git). Automated pipelines trigger automated builds and unit/integration tests. Code Commit → Automated Build → Automated Test	Automates the packaging, staging, and deployment of tested code into testing, staging, and production environments without manual friction. Build & Test → Staging → Automated Production Deploy

 **Remember:** CI = Code + Build + Test | CD = Release + Deploy to Environments.

18. Blue-Green, 19. Canary, & 20. High Availability ★★★★★

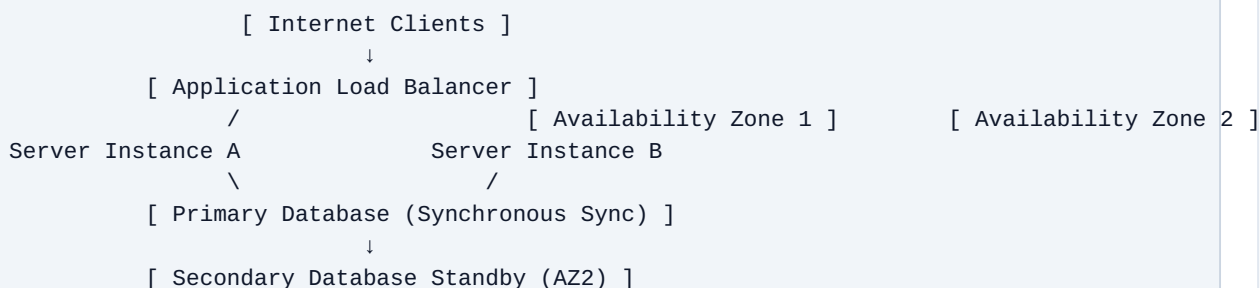
Modern Deployment Patterns

Pattern	Mechanism	Key Advantage
Blue-Green Deployment	Maintains two identical production environments: <i>Blue</i> (running live version) and <i>Green</i> (running new release). Once tested, traffic router flips 100% traffic from Blue to Green.	Instant zero-downtime cutover and near-instantaneous rollback if defects are detected (switch router back to Blue).

Canary Deployment	The new software release is exposed to a small canary group of users (e.g., 5%). After monitoring error rates, traffic is incrementally stepped up (5% → 25% → 50% → 100%).	Minimizes Blast Radius by validating stability under real production load before full-scale adoption.
--------------------------	---	---

High Availability (HA) Architecture Principles

HA systems are engineered to operate continuously without failure for long periods, eliminating **Single Points of Failure (SPOF)** through redundancy and automated failover across multiple Availability Zones (AZs).



ACCENTURE CLOUD ASSESSMENT MULTIPLE CHOICE QUESTIONS (Q1 – Q30)

Q1. Serverless Computing Model ★★★★★

A developer wants to deploy application code without provisioning or managing virtual machines or operating systems. The cloud provider automatically manages the underlying infrastructure. Which approach is most suitable?

- A. Traditional on-premises server
- B. Serverless computing
- C. Manual virtualization
- D. Dedicated physical server

✓ Correct Answer: B. Serverless computing

Option Analysis:

- **A ✗** — On-premises requires full in-house management of physical hardware and facilities.
- **B ✓** — Serverless computing abstracts all server, OS, and infrastructure management directly to the cloud provider.
- **C ✗** — Virtualization still requires configuring and managing guest operating systems and hypervisors.
- **D ✗** — Dedicated bare-metal servers require complete manual operational maintenance.

💡 *Key takeaway: Serverless ≠ No servers. It means you don't manage the servers.*

Q2. Function as a Service (FaaS) Paradigm

A function should execute automatically whenever a user uploads an image to cloud storage. Which model is most appropriate?

- A. FaaS (Function as a Service)
- B. IaaS (Infrastructure as a Service)
- C. Block storage
- D. CDN (Content Delivery Network)

✓ **Correct Answer: A. FaaS**

Option Analysis:

- **A ✓** — FaaS is specifically designed for event-driven compute execution triggered by cloud events (e.g., S3 ObjectCreated).
- **B ✗** — IaaS provides raw virtual machines which must run continuously.
- **C ✗** — Block storage provides raw block volumes (e.g., AWS EBS) for VMs.
- **D ✗** — CDN caches and distributes static/dynamic assets globally at edge locations.

Q3. Serverless vs FaaS Relationship

Which statement is most accurate regarding Serverless and FaaS?

- A. Serverless means there are literally no servers in existence
- B. FaaS is one common serverless compute model
- C. FaaS always requires physical server management by developers
- D. Serverless is identical to purchasing collocated physical servers

✓ **Correct Answer: B. FaaS is one common serverless model**

Option Analysis:

- **A ✗** — Servers exist; they are simply operated by the cloud vendor.
- **B ✓** — Serverless is the broader umbrella category, and FaaS represents its core event-driven compute subset.
- **C ✗** — FaaS completely abstracts server administration.
- **D ✗** — Collocation is the polar opposite of cloud serverless models.

Q4. Microservices Architecture Characteristics ★★★★★

An e-commerce application has separate services for Payment, Orders, Users, and Notifications. Each service can be developed and deployed independently. Which architecture is this?

- A. Monolithic architecture
- B. Microservices architecture
- C. Peer-to-peer architecture
- D. Batch architecture

✓ **Correct Answer: B. Microservices architecture**

Option Analysis:

- **A ✗** — Monolith packages all functional modules into a single coupled runtime.
- **B ✓** — Microservices decompose business logic into independently developable, deployable, and scalable units.
- **C ✗** — Peer-to-peer (P2P) is a decentralized networking protocol without central clients/servers.
- **D ✗** — Batch architecture processes non-interactive bulk jobs sequentially.

Q5. Advantages of Microservices

Which is a major advantage of microservices?

- A. Every service must always be deployed together simultaneously
- B. Individual services can often be deployed and scaled independently
- C. It completely eliminates all network communication
- D. It strictly mandates only one shared database for all services

✓ **Correct Answer: B. Individual services can often be deployed and scaled independently**

Option Analysis:

- **A** ✗ — Co-deployment is a drawback of monolithic architectures.
- **B** ✓ — Granular operational control over individual microservices enables targeted scaling and continuous deployment.
- **C** ✗ — Microservices inherently increase network communication via REST/gRPC.
- **D** ✗ — Microservices encourage the database-per-service pattern.

Q6. Challenges in Microservices Architectures

Which is a common challenge associated with microservices?

- A. Distributed-system complexity
- B. Elimination of all networking layers
- C. Inability to scale individual services
- D. Inability to deploy services independently

✓ **Correct Answer: A. Distributed-system complexity**

Option Analysis:

- **A** ✓ — Handling distributed transactions, inter-service networking latency, distributed tracing, and partial system failures introduces significant operational complexity.
- **B, C, D** ✗ — These statements are incorrect descriptions of microservice capabilities.

Q7. Role of API Gateway ★★★★★

A mobile application sends requests to an API Gateway, which routes requests to User, Order, and Payment services. What is the API Gateway acting as?

- A. Central entry point for APIs
- B. Bare-metal physical server
- C. Database storage engine
- D. Operating system CPU scheduler

✓ **Correct Answer: A. Central entry point for APIs**

Option Analysis:

- **A** ✓ — An API Gateway acts as the single boundary reverse proxy receiving client traffic and routing it appropriately.
- **B, C, D** ✗ — These refer to low-level hardware or internal subsystem components.

Q8. Functional Capabilities of an API Gateway

Which function can commonly be provided by an API Gateway?

- A. Request routing and rate limiting
- B. Increasing physical server RAM dynamically
- C. Replacing all relational and NoSQL databases
- D. Manufacturing hardware servers

✓ **Correct Answer: A. Request routing and rate limiting**

Option Analysis:

- **A** ✓ — Standard API Gateway responsibilities include reverse proxy routing, token validation, rate limiting, and request transformation.
- **B, C, D** ✗ — Outside the scope of API Gateway software architecture.

Q9. Cloud Monitoring Concepts ★★★★★

A cloud administrator wants to know whether CPU utilization has exceeded 90%. Which capability is most relevant?

- A. Monitoring
- B. Backup
- C. Encryption
- D. Authentication

✓ **Correct Answer: A. Monitoring**

Option Analysis:

- **A** ✓ — Monitoring tracks quantitative telemetry data (CPU, RAM, latency) against operational thresholds.
- **B** ✗ — Backup handles point-in-time state replication.
- **C** ✗ — Encryption handles cryptography and data protection.
- **D** ✗ — Authentication verifies client/user identity.

Q10. System Event Logging

An application records events such as "User login failed", "Payment request received", and "Database connection error". What is this information called?

- A. Logs
- B. Subnets
- C. Hypervisors
- D. Containers

✓ **Correct Answer: A. Logs**

Option Analysis:

- **A** ✓ — Logs are time-stamped textual event streams generated by processes to provide historical audit trails.
- **B** ✗ — Subnets are IP address partitions within virtual networks.
- **C** ✗ — Hypervisors (VMMs) create and run virtual machines.
- **D** ✗ — Containers are isolated application runtime environments (e.g., Docker).

Q11. Monitoring vs Logging Distinction

Which statement correctly differentiates monitoring from logging?

- A. Monitoring tracks system health/metrics, while logging records events
- B. Monitoring only stores backups
- C. Logging automatically increases CPU capacity
- D. Monitoring and logging are completely identical functions

✓ **Correct Answer: A. Monitoring tracks system health/metrics, while logging records events**

Option Analysis:

- **A** ✓ — Correct: Monitoring focuses on aggregated performance metrics over time; logging records detailed historical event logs.
- **B, C, D** ✗ — Inaccurate definitions of observability disciplines.

Q12. Cloud Backup Fundamentals ★★★★★

A company regularly creates copies of its database so that it can restore data after accidental deletion. What is this?

- A. Backup
- B. Load balancing
- C. CDN
- D. API Gateway

✓ **Correct Answer: A. Backup**

Option Analysis:

- **A** ✓ — Backups are secondary data snapshots created expressly for restoration after data corruption or loss.
- **B** ✗ — Load balancers distribute network traffic across compute nodes.
- **C** ✗ — CDNs optimize edge caching.
- **D** ✗ — API Gateways handle API routing and security policies.

Q13. Disaster Recovery Scenario ★★★★★

A company's primary data center is destroyed by a natural disaster. The company activates another environment in a secondary region to restore its application. What concept is this?

- A. Disaster Recovery
- B. Authentication
- C. CDN
- D. Containerization

✓ **Correct Answer: A. Disaster Recovery**

Option Analysis:

- **A** ✓ — Disaster Recovery (DR) entails the complete operational workflow of restoring applications and infrastructure following a catastrophic failure.
- **B, C, D** ✗ — Unrelated architectural concepts.

Q14. Backup vs Disaster Recovery Relationship

Which statement is correct regarding Backup and Disaster Recovery?

- A. Backup is a copy of data; disaster recovery is the broader process/strategy for restoring systems and services after disruption
- B. Backup and DR always mean exactly the same thing
- C. DR only means encrypting data at rest
- D. Backup is used exclusively for computer networking

✓ **Correct Answer: A. Backup is a copy of data; disaster recovery is the broader process/strategy**

Option Analysis:

- **A ✓** — Backup provides the raw recoverable dataset; DR defines the architecture, failover execution, RTO/RPO objectives, and business continuity strategy.
- **B, C, D ✗** — Factually inaccurate.

Q15. Recovery Point Objective (RPO) ★★★★★

A company states: "After a disaster, we can tolerate losing at most 30 minutes of recently created data." Which metric represents this requirement?

- A. RTO (Recovery Time Objective)
- B. RPO (Recovery Point Objective)
- C. SLA (Service Level Agreement)
- D. Latency

✓ **Correct Answer: B. RPO**

Option Analysis:

- **A ✗** — RTO defines permissible downtime before services must be back online.
- **B ✓** — RPO specifies acceptable data loss expressed as a duration back in time.
- **C ✗** — SLA is the overall contractual service agreement between vendor and client.
- **D ✗** — Latency measures network propagation or compute delay.

Q16. Recovery Time Objective (RTO) ★★★★★

A company requires its online banking application to be restored within 15 minutes after a major failure. Which metric represents this requirement?

- A. RPO
- B. RTO
- C. Bandwidth
- D. Throughput

✓ **Correct Answer: B. RTO**

Option Analysis:

- **A ✗** — RPO governs data loss limits.
- **B ✓** — RTO defines the maximum acceptable delay/downtime required to bring systems fully operational.
- **C, D ✗** — Networking capacity metrics.

Q17. 🔥 RPO vs RTO Comparative Pairing

Which pairing correctly matches RPO and RTO to their respective focus areas?

- A. RPO → Recovery time; RTO → Data loss
- B. RPO → Data loss; RTO → Recovery time
- C. RPO → Network speed; RTO → CPU usage
- D. RPO → Authentication; RTO → Authorization

✅ **Correct Answer: B. RPO → Data loss; RTO → Recovery time**

Option Analysis:

- **A ❌** — Inverted.
- **B ✅** — RPO (Recovery Point) = Data loss tolerance; RTO (Recovery Time) = System downtime tolerance.
- **C, D ❌** — Unrelated technical concepts.

Q18. Multi-Cloud Deployments ★★★★★

A company uses AWS for its application servers and Azure for its analytics workload. What strategy is this?

- A. Hybrid cloud
- B. Multi-cloud
- C. Private cloud only
- D. On-premises only

✅ **Correct Answer: B. Multi-cloud**

Option Analysis:

- **A ❌** — Hybrid cloud combines on-premises / private cloud infrastructure with a public cloud.
- **B ✅** — Utilizing two or more independent public cloud vendors (AWS + Azure) constitutes a Multi-Cloud strategy.
- **C, D ❌** — Excludes public cloud infrastructure.

Q19. Cloud Bursting Architecture

A company's application normally runs in its private cloud. During major traffic spikes, additional workloads are temporarily deployed to a public cloud. What is this called?

- A. Cloud bursting
- B. Blue-green deployment
- C. Authentication
- D. Containerization

✅ **Correct Answer: A. Cloud bursting**

Option Analysis:

- **A ✅** — Cloud bursting dynamically provisions public cloud resources when on-premises private datacenter compute capacity hits capacity limits.
- **B ❌** — Software deployment cutover strategy.
- **C ❌** — User verification.
- **D ❌** — Packaging software into lightweight runtimes.

Q20. Infrastructure as Code (IaC) Tooling ★★★★★

An administrator defines cloud infrastructure using Terraform configuration files and uses those files to create networks and servers. Which concept is being used?

- A. Infrastructure as Code
- B. Manual provisioning
- C. CDN
- D. DNS

✓ Correct Answer: A. Infrastructure as Code

Option Analysis:

- **A** ✓ — Terraform is an industry-standard Infrastructure as Code tool that provisions cloud infrastructure from declarative configuration files.
- **B** ✗ — Manual provisioning uses the management web console.
- **C** ✗ — CDN caches web assets.
- **D** ✗ — DNS resolves domain names to IP addresses.

Q21. Benefits of IaC

Which is a major benefit of Infrastructure as Code?

- A. Repeatable, version-controlled, and consistent infrastructure
- B. Completely eliminates all ongoing cloud billing costs
- C. Removes the need for all cybersecurity controls
- D. Physically prevents all hardware component failures

✓ Correct Answer: A. Repeatable, version-controlled, and consistent infrastructure

Option Analysis:

- **A** ✓ — IaC codifies environments into git repositories, guaranteeing exact environment replication and eliminating configuration drift.
- **B, C, D** ✗ — Factually impossible claims.

Q22. Continuous Integration (CI) Workflow ★★★★★

A developer pushes code to a shared repository. An automated pipeline builds the application and runs automated tests. Which practice is primarily being demonstrated?

- A. Continuous Integration
- B. Disaster Recovery
- C. Cloud Bursting
- D. CDN

✓ Correct Answer: A. Continuous Integration

Option Analysis:

- **A** ✓ — CI is the automated process of committing code, triggering builds, and executing automated test suites early.
- **B** ✗ — DR restores services after disruption.
- **C** ✗ — Cloud bursting manages hybrid capacity.
- **D** ✗ — Edge distribution network.

Q23. Continuous Delivery / Deployment (CD)

After automated testing succeeds, the pipeline automatically deploys the application into production. Which concept is most closely associated with this?

- A. Continuous Delivery / Deployment
- B. Object storage
- C. Authentication
- D. Virtualization

✓ **Correct Answer: A. Continuous Delivery / Deployment**

Option Analysis:

- **A** ✓ — CD extends CI by automating the release and deployment of validated binaries to staging and production targets.
- **B, C, D** ✗ — Unrelated infrastructure primitives.

Q24. Blue-Green Deployment Strategy ★★★★★

A company maintains two production environments: Blue (current version) and Green (new version). After testing Green, all traffic is switched from Blue to Green. What deployment strategy is this?

- A. Canary deployment
- B. Blue-green deployment
- C. Rolling backup
- D. Cloud bursting

✓ **Correct Answer: B. Blue-green deployment**

Option Analysis:

- **A** ✗ — Canary rolls out to a small percentage incrementally.
- **B** ✓ — Blue-green provisions two parallel environments and performs an all-at-once traffic switch via router/load balancer.
- **C, D** ✗ — Irrelevant options.

Q25. Canary Deployment Strategy ★★★★★

A company sends a new application version to only 5% of users. If no problems occur, it gradually increases traffic to the new version. What is this?

- A. Blue-green deployment
- B. Canary deployment
- C. Disaster recovery
- D. Backup

✓ **Correct Answer: B. Canary deployment**

Option Analysis:

- **A** ✗ — Blue-green maintains two entire environments with a sudden cutover.
- **B** ✓ — Canary tests releases against small fractions of live traffic (e.g., 5% → 25% → 100%) to mitigate blast radius.
- **C, D** ✗ — Business continuity terms.

Q26. Blue-Green vs Canary Deployment Comparison

Which statement correctly distinguishes Blue-Green from Canary deployment?

- A. Blue-green gradually sends 1% of traffic while canary always switches 100% immediately
- B. Blue-green uses two full environments; canary gradually exposes a new version to a subset of users
- C. Both are database backup methods
- D. Both are physical storage models

✓ **Correct Answer: B. Blue-green uses two full environments; canary gradually exposes a new version**

Option Analysis:

- **A** ✗ — Reverses the actual mechanisms.
- **B** ✓ — Correct: Blue-Green switches whole traffic between two identical setups, whereas Canary phases in traffic incrementally.
- **C, D** ✗ — Inaccurate categorizations.

Q27. High Availability (HA) Best Practices ★★★★★

Which architecture provides the highest degree of availability?

- A. One server with no backup
- B. Multiple servers across failure-isolated locations with health checks and failover
- C. One database with no backup
- D. One physical network connection

✓ **Correct Answer: B. Multiple servers across failure-isolated locations with health checks and failover**

Option Analysis:

- **A, C, D** ✗ — Single Points of Failure (SPOF) that guarantee downtime during failures.
- **B** ✓ — Redundancy across multiple Availability Zones combined with automated health checks ensures seamless failover and high availability.

Q28. Single Point of Failure (SPOF)

A web application depends on one server. If that server fails, the entire application becomes unavailable. What is this server called?

- A. Single point of failure (SPOF)
- B. CDN
- C. Load balancer
- D. API Gateway

✓ **Correct Answer: A. Single point of failure (SPOF)**

Option Analysis:

- **A** ✓ — A SPOF is any single non-redundant component whose disruption causes total system outage.
- **B, C, D** ✗ — Architectural tools used to improve performance, routing, or availability.

Q29. 🔥 Mixed Architectural Scenario

An organization has the following requirements:

- Application normally runs in a private cloud.
- Extra capacity is obtained from a public cloud during peak traffic.
- Infrastructure is created using Terraform.
- Application is deployed gradually to 5% of users before full rollout.

Which combination of concepts is correct?

- A. Cloud bursting + IaC + Canary deployment
- B. Multi-cloud + Backup + Blue-green deployment
- C. Serverless + RPO + Authentication
- D. CDN + RTO + Block storage

✅ **Correct Answer: A. Cloud bursting + IaC + Canary deployment**

Requirement Breakdown:

- *Private cloud + public during peaks* → **Cloud bursting**
- *Terraform automation* → **Infrastructure as Code (IaC)**
- *5% incremental user rollout* → **Canary deployment**
- **B, C, D** ❌ — Fail to align with the specified criteria.

Q30. 🔥 Accenture-Level Mixed Enterprise Scenario

An e-commerce company operates an architecture where Users connect via an API Gateway to Microservices backed by Cloud Databases. During a festival:

- Traffic increases dramatically; Auto Scaling adds instances while the Load Balancer distributes traffic.
- Monitoring detects high CPU usage.
- The company requires an RTO of 30 minutes and an RPO of 5 minutes.

Which statement is correct?

- A. RTO represents maximum acceptable data loss and RPO represents recovery time
- B. Auto scaling distributes traffic while the load balancer adds servers
- C. RTO represents the target recovery time, while RPO represents the acceptable data-loss window
- D. Monitoring and logging are exactly the same thing

✅ **Correct Answer: C. RTO represents the target recovery time, while RPO represents the acceptable data-loss window**

Option Analysis:

- **A** ❌ — Inverted: RTO is time/downtime; RPO is data loss.
- **B** ❌ — Inverted: Load Balancers distribute traffic across instances; Auto Scaling adds/removes instances.
- **C** ✅ — Correct: RTO (30 min) = maximum time to recover operations; RPO (5 min) = maximum tolerable data loss window.
- **D** ❌ — Monitoring tracks metrics/health; logging records historical event details.